

Revisiting the Foundations of Dynamic Multi-Layer Algebra 3.5

Gergely Gembela Gergely Mezei Ferenc A. Somogyi
Áron Furmann

*Department of Automation and Applied Informatics
Budapest University of Technology and Economics*

{gergelygembela, gmezei, somogyi.ferenc, aron.furmann}@aut.bme.hu

Abstract. Multi-level metamodeling (MLM) is a promising alternative to traditional two-level modeling approaches as it enables classification and instantiation across multiple abstraction levels. Many existing solutions remain constrained by rigid level boundaries, limited runtime adaptability, or limited support for semantic evolution. The Dynamic Multi-Layer Algebra (DMLA) addresses these challenges through a formal framework in which structural elements, relations, behavioral definitions, and validation mechanisms are represented within a single executable model space.

After more than a decade of development, DMLA has reached a stage where the primary focus is shifting from core language design toward systematic testing, practical use cases, and the identification of conceptual and usability issues arising in real-world applications. This shift has resulted in several substantial refinements of the framework. This paper presents the current interpretation of the fundamental concepts underlying DMLA models and describes the updated principles. We intend for this paper to serve as a consolidated reference point for future development, evaluation, and application of the framework.

Keywords: Multi-Level Modeling; Metamodeling; DMLA; Systems Engineering; Dynamic Multi-Layer Algebra

1 Introduction

The Dynamic Multi-Layer Algebra (DMLA) is a framework that provides both the theoretical background and a software stack for Multi-level modeling (MLM). DMLA models represent structural and behavioral model elements along with their relationships. With recent developments, the DMLA framework has become mature enough to be applied in practical scenarios. While preparing DMLA for integration with external systems, we have reached a new milestone: DMLA 3.5. This paper aims to present the current state of DMLA’s theoretical background by summarizing the main concepts and their connections.

Section 2 provides theoretical background and related work; Sections 3-3.6 describe the structural model elements, refinement, type system, possible annotation variants, and finally the customizable validation logic. Section 4 describes future work and concludes the paper.

The syntax and semantics are illustrated with examples of the *warehouse* model, also available as a *dev container* runnable either locally in VS Code or via GitHub Codespaces [1].

2 Background and Related Work

MLM is a paradigm aiming to reduce *accidental complexity* [2] naturally introduced by the structure of classic two-level models by strictly separating the schema (metamodel) and instance (model) layers. This separation often introduces unnecessary overhead in model maintenance and evolution. MLM combines the schema and instance, allowing an arbitrary number of abstraction levels connected by relations, constraints, and instantiations that can span multiple layers (also known as *deep instantiation*).

Applicability and tool support of multi-level modeling have been studied extensively in the Workshop on Multi-Level Modeling (MULTI) series. To enable comparison between approaches, the community has defined a set of benchmark challenges over the years, addressing problems such as engineering process management systems, product variant handling, and dynamically configurable multi-currency webshops [3, 4, 5]. The challenges highlight that the area is well-established and actively researched, with a wide range of proposed solutions.

Most multi-level modeling approaches rely on the notion of *potency* to control instantiation depth [6]. In these approaches, models are organized into explicit abstraction levels, and instantiation is typically restricted to adjacent levels, enforcing a strict hierarchical structure.

Level-blind approaches take a different direction: instantiation does not rely on explicit metalevel boundaries; it is defined directly between model elements. While levels may still be present implicitly, they are not treated as primary structuring elements. This removes the need to fix the number of abstraction levels in advance and allows instantiation chains to span across the model more freely.

DMLA follows this level-blind approach, as it fits well with the way models are often developed in practice: incrementally, with structure and details emerging over time rather than being fixed upfront [7].

When the DMLA modeling environment loads models that share a common root directory (a workspace), a long-lived workspace session is created that hosts the modeling space. Once models are loaded, the environment can process subsequent execution requests, creating separate execution sessions for each request. Execution sessions maintain an extended model space that exists only until their entry point (the modeled operation invoked externally) terminates.

3 Building Blocks

The Dynamic Multi-layer Algebra (DMLA) framework provides a formal mechanism for defining abstract concepts, enriching them with structured features,

and establishing relations among them within a unified model. The construction process follows a top-down paradigm, in which high-level structures are specified first and subsequently refined through the incremental introduction of detail. This approach ensures that consistency and formal rigor are preserved throughout the refinement process. At the same time, DMLA is designed to maintain a high degree of flexibility while supporting systematic validation at each stage of model evolution.

In this paper, we use a simplified solution of the warehouse challenge [5] to illustrate the mechanisms of DMLA. Fig. 1 shows a portion of the model. Entities are depicted as rounded rectangles; their operations and slots are shown in separate internal frames. Annotations are shown above their receivers; their names always start with an '@' symbol (e.g., @Type). Classifiers are always specified after a ':' symbol (e.g. **Refinement : Classifier**). Slots are displayed with alternating background shading to improve visual separation, as most slot specifications span multiple rows. For commonly used elements that form essential parts of DMLA models, we introduce special visual syntax elements. The '→|' symbol is used to denote when a metaslot is omitted by a slot definition, while slot specifications **written in bold** show final slots (as explained later). As the visual syntax evolves, more symbols will be introduced to improve clarity and brevity.

3.1 Entities

Entities are the fundamental building blocks of DMLA models. Similarly to classes in object-oriented programming (OOP), entities represent domain concepts at various levels of abstraction. An entity may define structure, behavior, and constraints through its associated model elements. More specifically, entities may contain *slots* and *operations*, and may receive *annotations*. Slots describe properties of the modeled concept, while operations define behavior, including validation logic and domain-specific functionality. Annotations provide a uniform mechanism to impose constraints or influence behavior, for example, type restrictions or cardinality constraints. While operations are specific to a given entity, annotations typically capture reusable semantics applicable across multiple entities. Annotations may not be attached to other annotations, but all other model elements can receive annotations.

In DMLA models, slots, operations, and annotations are modeled as entities, resulting in a uniform representation across model elements, and thus all elements participate in the same refinement and validation mechanisms.

3.2 Classification and Refinement

Each entity has a unique *classifier*, which serves as a template defining its structural elements, including slots, operations, and annotations. The relationship between an entity and its classifier is referred to as *refinement*. A sequence of such relationships forms a refinement chain, which may be of arbitrary length. In our example model, the *Product* entity is the starting point of multiple re-

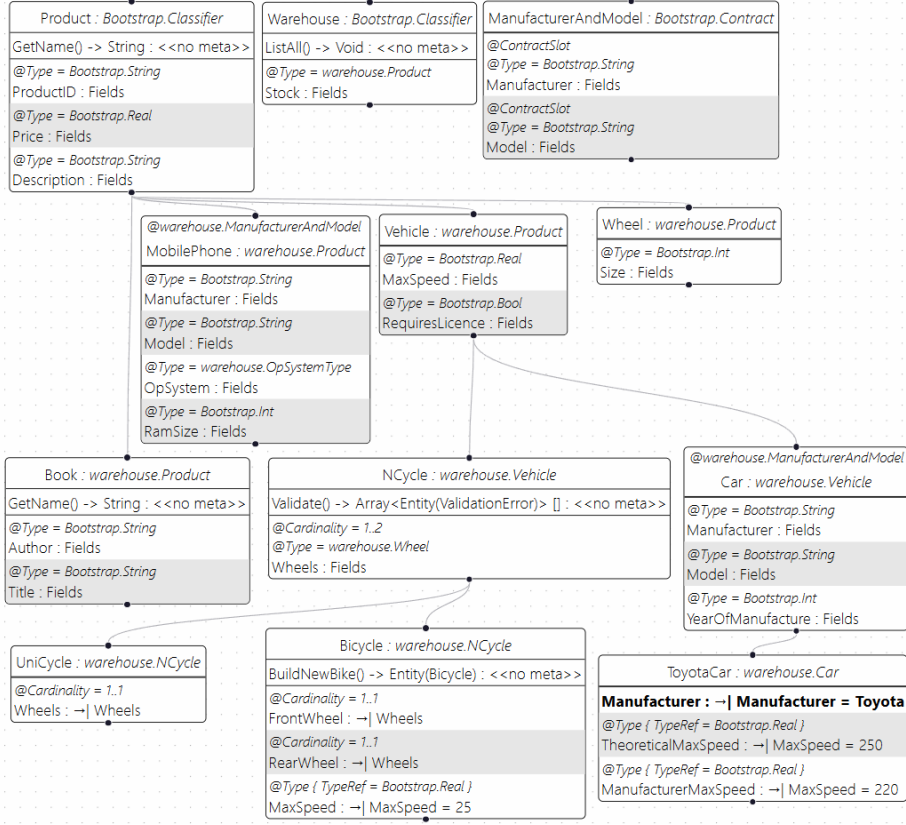


Figure 1: A DMLAVis visualization of a part of the warehouse demo project

finement chains. Refinements are depicted with curved lines connecting model elements. The curve always starts at the bottom border of the classifier and connects to the top border of the refinement.

Entities are required to conform to the structural constraints imposed by their classifiers, while refinements progressively concretize and specialize the concepts represented at higher levels of abstraction. At the top of this hierarchy, every entity is a (direct or indirect) refinement of the built-in *Base* entity, which is self-defined, i.e., it serves as its own classifier.

The refinement structure is strictly acyclic: loops are not permitted in the refinement chain, with the sole exception of the *Base* entity due to its self-referential definition. When an entity is refined, not only the entity itself but also its contained elements (slots, operations, and annotations) may be further refined in a consistent manner along the refinement chain.

3.3 Slots

Slots represent properties of modeled concepts (e.g., a *Product* has a *Price*). In entity definitions, slots are only explicitly declared when they are modified or specialized. Inherited slots that remain unchanged are defined implicitly: they are not listed in the definition but remain fully accessible and referenceable (the slot *Price* is not listed in entity *Wheel*, although wheels do have a price).

Slots accessible through the refinement chain may be further refined at any point, even skipping one or more intermediate refinement steps. For example, both the *Bicycle* and *ToyotaCar* entities can refine the *MaxSpeed* slot, even though it was defined further up on their meta-chain. To ensure consistency, no Entity may have two slots with the same name accessible at any level, including implicit slots inherited from its Classifier.

Refinements of slots may take several forms. A slot can be constrained by adding or modifying annotations or by assigning a value. In our example, the *UniCycle* entity restricts the cardinality of its *Wheels* slot to 1, and the *Bicycle* specifies the value of its *MaxSpeed* slot. If a slot has an assigned value, that value may be overridden in subsequent refinements (e.g., if we were to introduce a *RacingBicycle* entity, it could refine the *MaxSpeed* slot with a higher value).

Slots may also be explicitly omitted, meaning that the given feature (presented by the slot) is not available in the concept refinement chain below. An entity may mark a slot as omitted using the *Omit* annotation. Omitted slots become inaccessible and cannot be referenced further down the refinement chain.

A distinctive feature of DMLA is that slots may be structurally transformed during refinement. In particular, a slot may be divided into multiple slots to represent distinct aspects or fragments of the original feature. For example, the general *Wheels* slot of entity *NCycle* is refined into the *frontWheel* and *rearWheel* slots of the *Bicycle* entity. When a slot is overridden without the *new* keyword, the original slot is implicitly omitted, indicated with the ' $\rightarrow$ ' symbol preceding its metaslot specification (e.g., the *FrontWheel* and *RearWheel* slots consume the more general *Wheels* slot). In the case of slot division, the *new* keyword must be applied consistently (i.e., to all resulting slots) for the original slot to be preserved.

Slot division can also be used to rename a slot by creating a new slot from the original and omitting the original one. Introduction of entirely new slots is formalized as a special case of division from the universal *Base.Fields* slot. Consequently, adding new slots is only possible as long as *Base.Fields* has not been omitted.

3.4 The Type System

The DMLA type system distinguishes between primitive values and modeled entities. Primitive values can be of types (i) *Boolean*, (ii) *Integer*, (iii) *Float* or (iv) *String*. Modeled entities are further divided into static and dynamic entities. Static entities are included in the modeling space, and unless they are explicitly removed via dynamic modeling operations, they are persistent. In

contrast, dynamic entities are tied to execution sessions. They can be used to model structural details, operations may be executed on them, but they are only added to the extended modeling space. As a result, dynamic entities may only be referenced by dynamic entities. Dynamic entities may reference static entities as well, but static entities cannot reference dynamic ones. This also means that all slots and annotations of dynamic entities must also be dynamic. Dynamic entities are useful in scenarios such as passing structured parameters or reporting validation errors. It is important to note that modeling APIs provided by the DMLA runtime support creating both dynamic and static entities even at runtime.

3.5 Annotations

In DMLA, *annotations* carry metadata and are similar to annotations in Java or Kotlin, or attributes in .NET languages. *Constraints* are annotations with a special operation called *Check*, which is used to evaluate whether the defined logical condition holds for a given model element. Through this mechanism, the framework determines whether the constraint is satisfied in the context of the annotated entity or slot.

1. The *Omit* constraint can be applied to a slot. It indicates that the annotated slot cannot be referenced further down the refinement chain. *Omit* is used implicitly by the framework when a slot is defined without the *new* keyword (in this case, the origin slot is omitted).
2. The *Type* constraint annotation specifies the type of a slot's value. The type can be either a primitive or a modeled type. In the case of modeled types, type compatibility is checked along the refinement chain. For example, the *Product.ProducID* slot will have a String type at any point in the chain. *Vehicles* that refine the *Wheels* slot of the *Vehicle* entity may further restrict the type of wheels they accept, but only to refinements of the *Wheel* entity.
3. The *Cardinality* constraint defines how many fragments a slot can be divided into, or how many values it can hold (e.g., an *NCycle* may have one or two *Wheels*; the *Bicycle* entity divides this into the *FrontWheel* and *RearWheel* slots, and by restricting their cardinality, it essentially ensures they cannot be divided any further). Cardinality has a minimum and a maximum value. The maximum can be set to -1, which means that there is no upper limit for the multiplicity.
4. The *Final* constraint indicates that the receiver (i.e., an annotated entity or slot) represents an *object* that cannot be subject to further instantiations, essentially terminating the meta-chain. When applied to an entity, all slots of the entity with no value assigned are treated as if they were omitted. All slots with values assigned need to hold final values (primitives or references to final entities). The **Manufacturer** slot of the *ToyotaCar*

entity shown in Fig. 1 is final, as the manufacturer of Toyota cars is known to be *Toyota* for all refinements.

5. The *Contract* constraint enables partial validation of model elements by requiring an entity to satisfy a defined set of structural conditions. Similar to interfaces in object-oriented programming, contracts specify required structural properties without introducing additional refinement relationships. In this way, they allow structural constraints to be imposed independently of the refinement hierarchy. A contract defines a set of slots, which may be annotated with *ContractSlot*. The entity must implicitly or explicitly contain a corresponding slot for each contract slot, matched by name. Furthermore, the entity's slots must carry all constraints and annotations defined on the corresponding contract slots (or their refinements), and the constraints must also be satisfied by the entity's slots. The *ManufacturerAndModel* contract shown in Fig. 1 has two contract slots. Applied to the *MobilePhone* entity, it enforces that *MobilePhone* and all its refinements have slots with the names *Manufacturer* and *Model* that have the types prescribed in *ManufacturerAndModel*'s slot's annotations.

3.6 Model Validation

Consistency in DMLA models is ensured through an explicit validation mechanism. Every entity has a *Validate* operation, which is used to check the correctness of its refinement. Validation may either return a collection of errors or an empty list in case it succeeds. Validation checks are applied automatically by the framework, but they can also be called manually.

During refinements, the *Validate* operation can be overridden; however, all *Validate* operations along the entire refinement chain are always checked, thus constraints can only become stricter, not more relaxed. Validation can also be influenced by constraints. Similar to *Validate*, *Check* returns a list of errors. Constraints applied to entities are automatically checked whenever the entity is validated.

The validation rules defined within an entity apply only to its refinements, not to the entity itself. However, constraints are checked even at the level at which they are applied. For example, if a type annotation is added to a slot and a value is also assigned, the validation immediately verifies the value at that level (e.g., the *Product.Description* slot is ensured to be a string even when the *Product* entity is checked).

4 Conclusion and Future Work

DMLA has undergone several incremental changes since its initial design, and with each iteration, its range of potential use cases has widened. DMLA 3.5 represents a significant step forward in the practical applicability of multi-level modeling by providing a flexible, level-blind framework that unifies structural, behavioral, and validation aspects within a single executable model space. With

recent developments eliminating the need for complicated setup procedures or intricate familiarity with the system and its dependencies, the DMLA framework is becoming ready for real-world multi-level modeling tasks. With multiple practical use cases under development, DMLA is expected to be increasingly shaped by the requirements of applications. Further development is expected to be driven mainly by integration requirements, testing, and usability feedback. These experiences will help us identify the remaining conceptual issues and guide future refinements of both the formal model and the supporting tooling. Future work will also focus on comparing DMLA with other (level-blind) multi-level modeling tools mainly based on challenges of the MULTI workshop series.

References

- [1] G. Gembela et al., “Archived github repository publishing a dev container as sandbox for a multi 2023 warehouse challenge example in dmla.” 2026. <https://github.com/dmlaorg/revisiting-foundations-aacs2026>.
- [2] T. Kühne and C. Atkinson, “Reducing accidental complexity in domain models,” *Software & Systems Modeling*, vol. 7, pp. 345–359, Jul 2008.
- [3] J. P. A. Almeida, A. Rutle, M. Wimmer, and T. Kuhne, “The multi process challenge,” in *2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, pp. 164–167, 2019.
- [4] G. Mezei, T. Kühne, V. Carvalho, and B. Neumayr, “The multi collaborative comparison challenge,” in *2021 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, pp. 495–496, 2021.
- [5] T. Kühne and M. A. Jeusfeld, “The multi warehouse challenge,” in *2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, pp. 699–702, 2023.
- [6] T. Kühne, “Exploring potency,” in *Proceedings of the 21th ACM/IEEE International Conference on Model Driven Engineering Languages and Systems*, MODELS ’18, (New York, NY, USA), pp. 2–12, Association for Computing Machinery, 2018.
- [7] M. Jeusfeld, G. Mezei, and S. Bácsi, “Deeptelos and DMLA – A contribution to the MULTI 2022 collaborative comparison challenge,” in *Proceedings of the ACM/IEEE 25th International Conference on MODELS: Companion Proceedings*, (New York, NY, USA), ACM, 2022.